

KNN

The K-Nearest Neighbours (KNN) algorithm is one of the simplest supervised machine learning algorithms that is used to solve both classification and regression problems.

KNN is also known as an instance-based model or a lazy learner because it doesn't construct an internal model.

For classification problems, it will find the k nearest neighbors and predict the class by the majority vote of the nearest neighbors.

For regression problems, it will find the k nearest neighbors and predict the value by calculating the mean value of the nearest neighbors.

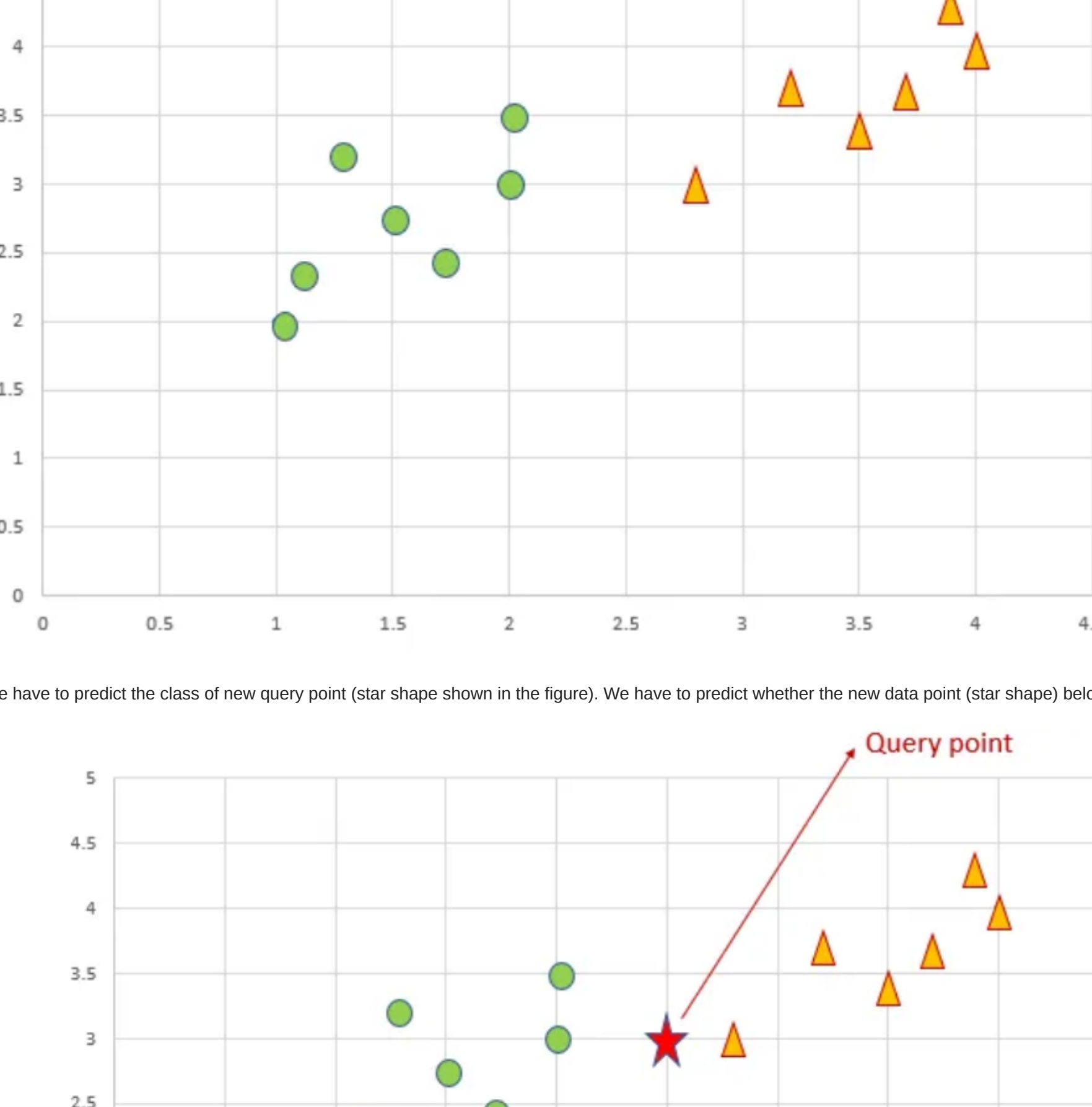
K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data.

The main concept behind KNN is Birds of same feather flock together

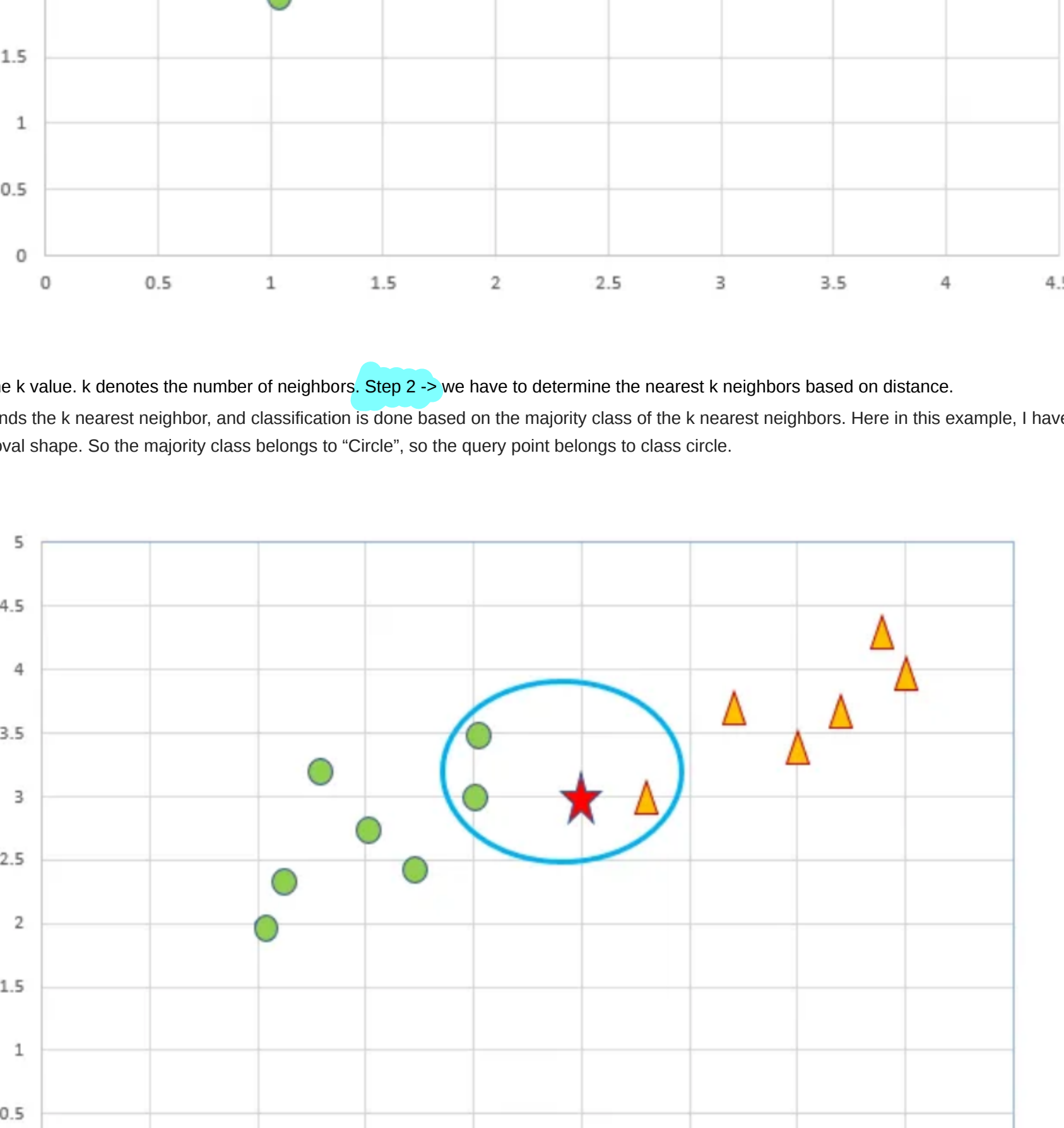
KNN Classification

Let's learn how to classify data using the knn algorithm. Suppose we have two classes circle and triangle.

Below is the representation of data points in the feature space.

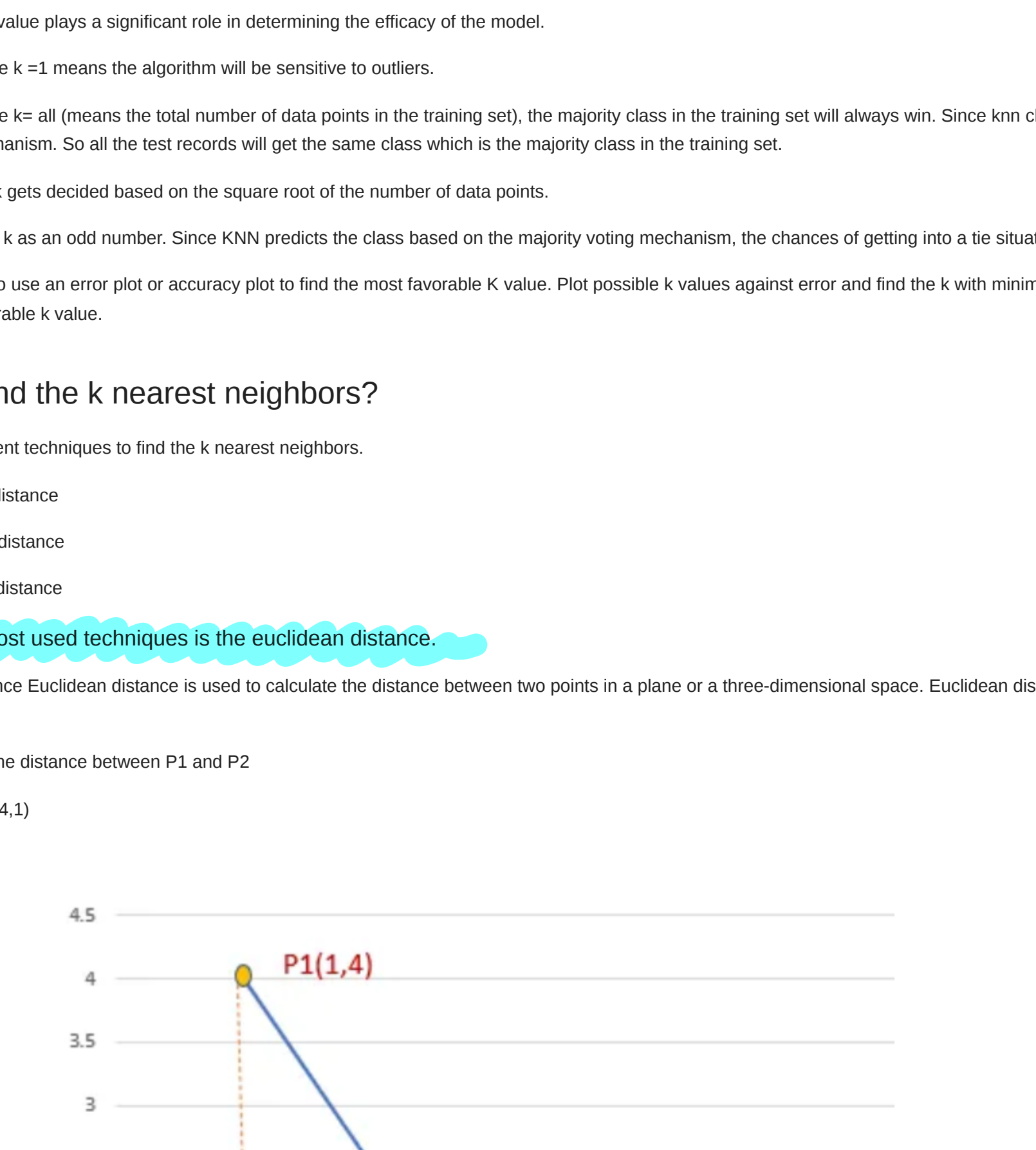


Now we have to predict the class of new query point (star shape shown in the figure). We have to predict whether the new data point (star shape) belongs to class circle or triangle.



Step 1 -> we have to determine k value. k denotes the number of neighbors. Step 2 -> we have to determine the nearest k neighbors based on distance.

This algorithm finds the k nearest neighbor, and classification is done based on the majority class of the k nearest neighbors. Here in this example, I have shown the nearest neighbors inside the blue oval shape. So the majority class belongs to "Circle", so the query point belongs to class circle.



Now, comes the important point. 1. How to find the optimum k value? 2. How to find the k nearest neighbors?

How to find the optimum k value?

Choosing the k value plays a significant role in determining the efficacy of the model.

1. If we choose k=1 means the algorithm will be sensitive to outliers.
2. If we choose k= all (means the total number of data points in the training set), the majority class in the training set will always win. Since knn classifies class based on majority voting mechanism. So all the test records will get the same class which is the majority class in the training set.
3. Generally, k gets decided based on the square root of the number of data points.
4. Always use k as an odd number. Since KNN predicts the class based on the majority voting mechanism, the chances of getting into a tie situation will be minimized.
5. We can also use an error plot or accuracy plot to find the most favorable K value. Plot possible k values against error and find the k with minimum error and that k value is chosen as the favorable k value.

How to find the k nearest neighbors?

There are different techniques to find the k nearest neighbors.

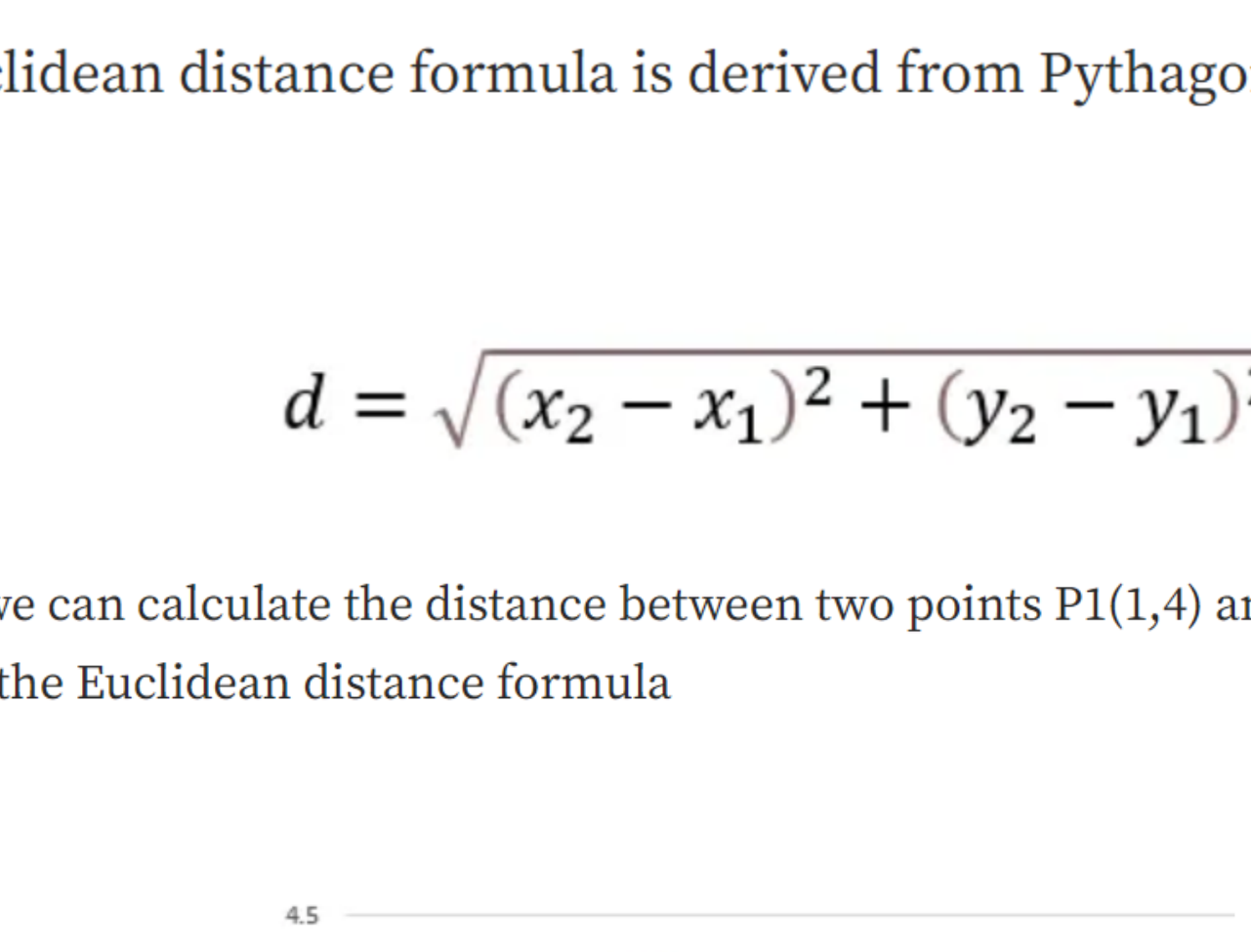
1. Euclidean distance
2. Manhattan distance
3. Minkowski distance

One of the most used techniques is the euclidean distance.

Euclidean distance Euclidean distance is used to calculate the distance between two points in a plane or a three-dimensional space. Euclidean distance is based on the Pythagoras theorem.

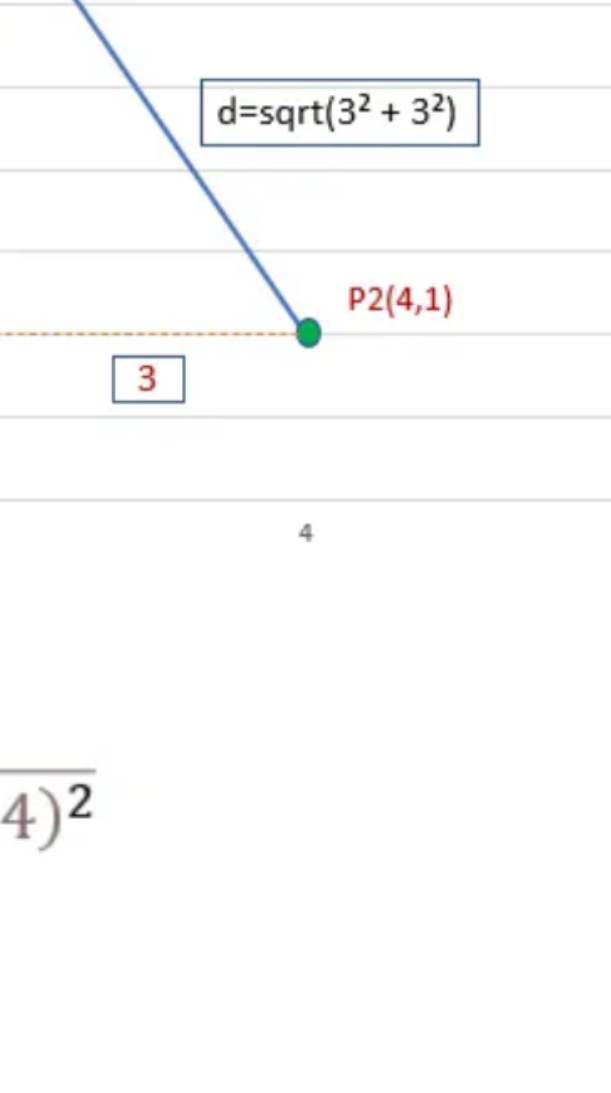
Let's calculate the distance between P1 and P2

P1(1,4) and P2(4,1)



Pythagoras theorem

In any right-angled triangle, the square of the hypotenuse(longest side of a triangle) is equal to the sum of the other two sides of the triangle.



$$a^2 + b^2 = c^2$$

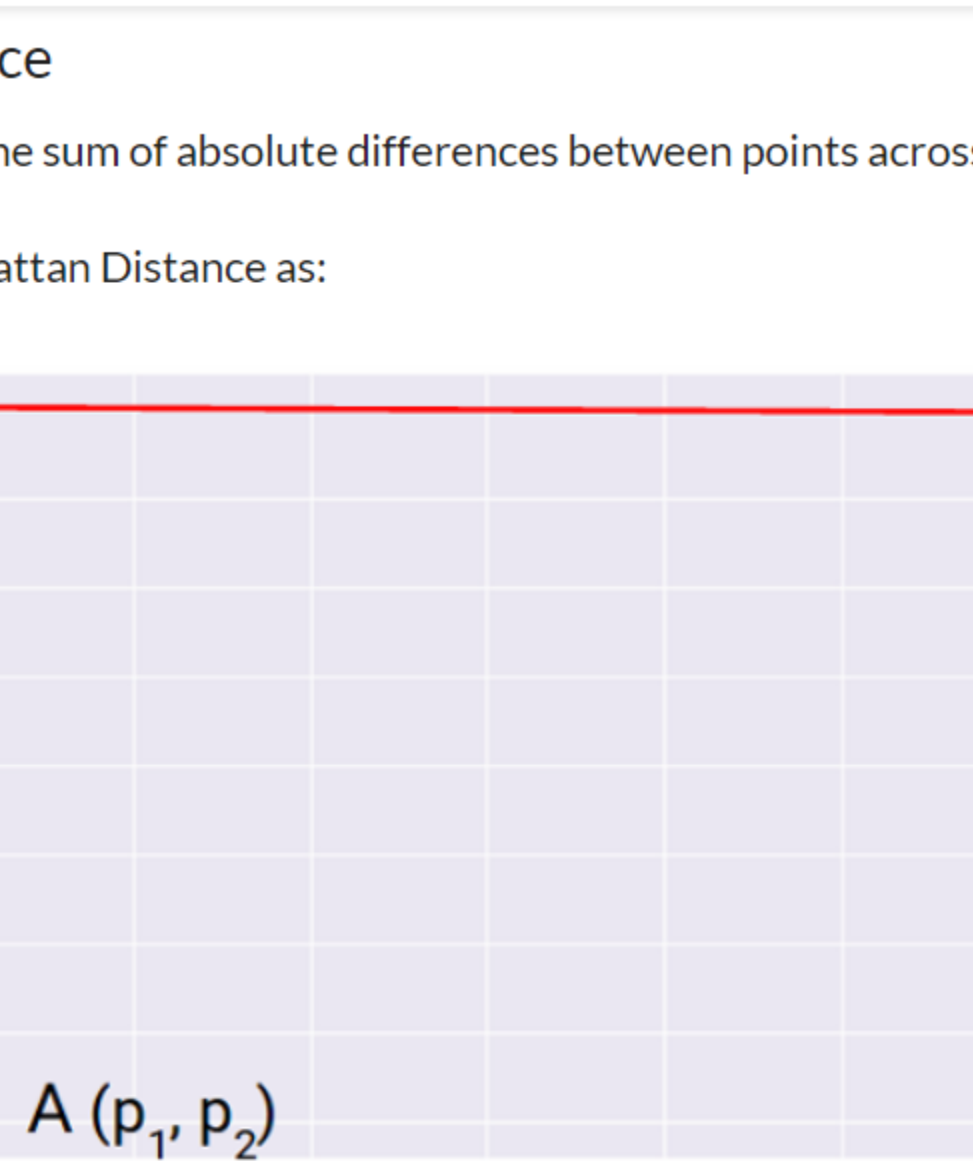
$$\rightarrow c^2 = a^2 + b^2$$

$$\rightarrow c = \sqrt{a^2 + b^2}$$

The Euclidean distance formula is derived from Pythagoras theorem

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Now we can calculate the distance between two points P1(1,4) and P2(4,1) using the Euclidean distance formula



$$d = \sqrt{(4 - 1)^2 + (1 - 4)^2}$$

$$d = \sqrt{9 + 9}$$

$$d = 4.24$$

KNN algorithm calculates the distance of all data points from the query points using techniques like euclidean distance. Then, it will select the k nearest neighbors. Then based on the majority voting mechanism, knn algorithm will predict the class of the query point. Important point to remember: The distance formula is highly dependent on how dimensions are measured. To ensure all dimensions have a similar scale, we normalize the data on all dimensions.

We will first import the required libraries. I will be using the SciPy library that contains pre-written codes for most of the distance functions used in Python:

```
1 # importing the library
2 from scipy.spatial import distance
3
4 # defining the points
5 point_1 = (1, 2, 3)
6 point_2 = (4, 5, 6)
7 point_1, point_2
```

library_and_dataset.py hosted with ❤ by GitHub

view raw

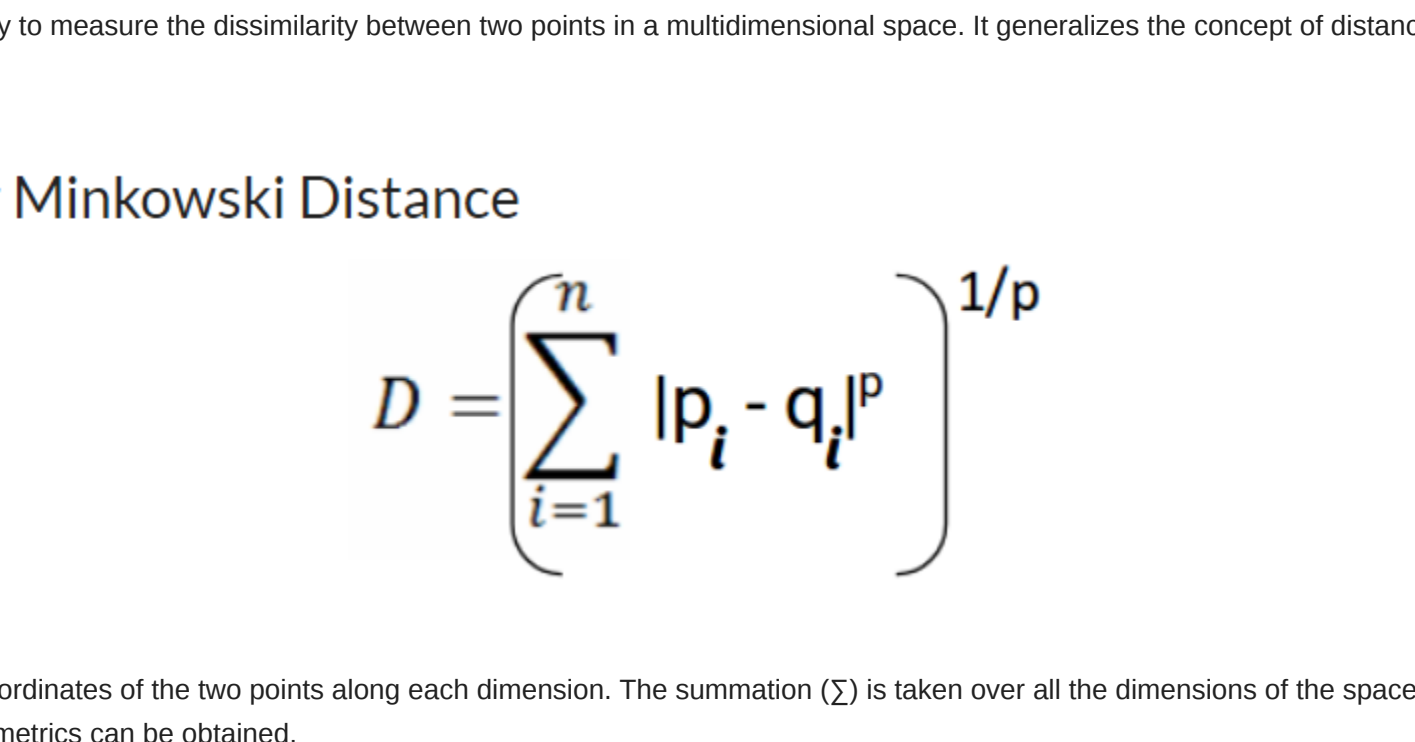
((1, 2, 3), (4, 5, 6))

These are the two sample points that we will be using to calculate the different distance functions.

Manhattan Distance

Manhattan Distance is the sum of absolute differences between points across all the dimensions.

We can represent Manhattan Distance as:



Formula for Manhattan Distance

Since the above representation is 2 dimensional, to calculate Manhattan Distance, we will take the sum of absolute distances in both the x and y directions. So, the Manhattan distance in a 2-dimensional space is given as:

$$d = |p_1 - q_1| + |p_2 - q_2|$$

And the generalized formula for an n-dimensional space is given as:

$$D_m = \sum_{i=1}^n |p_i - q_i|$$

Where,

- n = number of dimensions
- pi, qi = data points

Now, we will calculate the Manhattan Distance between the two points:

```
1 # computing the manhattan distance
2 manhattan_distance = distance.cityblock(point_1, point_2)
3 print('Manhattan Distance b/w', point_1, 'and', point_2, 'is: ', manhattan_distance)
```

manhattan_distance.py hosted with ❤ by GitHub

view raw

Manhattan Distance b/w (1, 2, 3) and (4, 5, 6) is: 9

Note that Manhattan Distance is also known as city block distance. SciPy has a function called *cityblock* that returns the Manhattan Distance between two points.

Minkowski distance

The Minkowski distance is a way to measure the dissimilarity between two points in a multidimensional space. It generalizes the concept of distance by introducing a parameter, denoted as p.

Formula for Minkowski Distance

$$D = \left(\sum_{i=1}^n |p_i - q_i|^p \right)^{1/p}$$

Here, pi and qi represent the coordinates of the two points along each dimension. The summation ($\sum$) is taken over all the dimensions of the space. By varying the value of the parameter p, different distance metrics can be obtained.

When p = 1, the Minkowski distance reduces to the Manhattan distance. The Manhattan distance, also known as the L1 distance or taxicab distance, calculates the absolute differences between the coordinates of the points along each dimension and sums them up:

$$D(p, q) = |p_1 - q_1| + |p_2 - q_2| + \dots + |p_n - q_n|$$

The Manhattan distance measures the distance required to travel along the grid-like streets of a city, where you can only move horizontally or vertically (no diagonal movements).

When p = 2, the Minkowski distance becomes the Euclidean distance. The Euclidean distance, also known as the L2 distance, calculates the square root of the sum of squared differences between the coordinates of the points along each dimension:

$$D(p, q) = \sqrt{(|p_1 - q_1|^2 + |p_2 - q_2|^2 + \dots + |p_n - q_n|^2)}$$

The Euclidean distance corresponds to the straight-line distance between two points in a Euclidean space. It is often used in geometric calculations and is familiar as the distance formula in two- or three-dimensional space.

By varying the value of p, you can obtain different distance metrics that lie between the Manhattan and Euclidean distances. For example, when p = 0.5, it is known as the Minkowski distance with a fractional power or the Chebyshev distance, which measures the maximum absolute difference along any dimension.

In summary, the Minkowski distance is a generalized distance metric that encompasses both the Manhattan distance (p = 1) and the Euclidean distance (p = 2) as specific cases. By adjusting the parameter p, you can obtain a continuum of distance metrics that interpolate between these two well-known distance measures.

Here, p represents the order of the norm. Let's calculate the Minkowski Distance formula of order 3:

```
1 # computing the minkowski distance
2 minkowski_distance = distance.minkowski(point_1, point_2, p=3)
3 print('Minkowski Distance b/w', point_1, 'and', point_2, 'is: ', minkowski_distance)
```

minkowski_distance.py hosted with ❤ by GitHub

view raw

Minkowski Distance b/w (1, 2, 3) and (4, 5, 6) is: 4.3267487109222245

The p parameter of the Minkowski Distance metric of SciPy represents the order of the norm. When the order(p) is 1, it will represent Manhattan Distance and when the order in the above formula is 2, it will represent Euclidean Distance.

Why KNN is known as an instance-based method or Lazy learner

KNN (K-Nearest Neighbors) is indeed known as an instance-based method or a lazy learner. This is because the KNN algorithm does not explicitly learn a model or a discriminative function from the training data, which define the model's behavior. Once the parameters are estimated, the training data itself is no longer needed for prediction. Some key characteristics of parametric algorithms are:

Instead, KNN stores all the training instances in memory during the training phase. These instances are considered as the "knowledge" or the training set. In other words, KNN memorizes the entire training dataset rather than extracting specific patterns or parameters from it.

During the prediction phase, when a new unlabeled instance is given, KNN searches for the K nearest neighbors (training instances) to the unlabeled instance based on a distance metric (e.g., Euclidean distance). The predicted class or value of the unlabeled instance is then determined based on the majority vote or weighted average of the labels of its K nearest neighbors.

The lazy learning nature of KNN arises from the fact that the algorithm defers the computational work (i.e., distance calculation) until the prediction phase. The actual learning or generalization happens at the time of prediction, where the algorithm looks up the closest neighbors from the stored training instances and makes predictions based on their labels.

This contrasts with eager learners or model-based algorithms that build an explicit representation of the training data during the training phase. These algorithms construct a model or a function that summarizes the relationship between the input features and the output labels, which can then be used for predictions without requiring the entire training dataset at prediction time.

In summary, KNN is considered a lazy learner or an instance-based method because it does not learn a model during training but rather memorizes the training instances, and it performs computations (such as distance calculations) at prediction time rather than during the training phase.

Summary

1. Load the data
2. Initialize K to your chosen number of neighbors
3. For each example in the data
 - 3.1 Calculate the distance between the query example and the current example from the data.
 - 3.2 Add the distance and the index of the example to an ordered collection
4. Sort the ordered collection of distances and indices from smallest to largest (in ascending order) by the distances
5. Pick the first K entries from the sorted collection
6. Get the labels of the selected K entries
7. If regression, return the mean of the K labels
8. If classification, return the mode of the K labels

Advantages

1. The algorithm is simple and easy to implement.
2. There's no need to build a model, tune several parameters, or make additional assumptions.
3. The algorithm is versatile: It can be used for classification, regression, and search (as we will see in the next section).

Disadvantages

The algorithm gets significantly slower as the number of examples and/or predictors/independent variables increase.

Parametric and non-parametric algorithms

Parametric and non-parametric algorithms are two broad categories of machine learning algorithms that differ in how they represent and generalize from the training data. Here's a detailed explanation of each category:

Parametric Algorithms:

Parametric algorithms make strong assumptions about the functional form of the relationship between the input features and the output labels. These algorithms learn a fixed number of parameters from the training data, which define the model's behavior. Once the parameters are estimated, the training data itself is no longer needed for prediction. Some key characteristics of parametric algorithms are:

Fixed model structure: Parametric algorithms have a predetermined model structure with a fixed number of parameters. Examples include linear regression, logistic regression, and linear SVM.

Assumption of data distribution: Parametric algorithms often assume a specific distribution for the data, such as a normal distribution.

Efficient training: Parametric algorithms typically have fast training times since they estimate a fixed set of parameters from the data.

Simplified model interpretation: Due to their fixed model structure, parametric algorithms offer straightforward interpretations of the learned parameters, such as coefficient values.

Parametric algorithms can perform well when the underlying assumptions about the data distribution and model structure hold true. However, they may struggle to capture complex relationships if the assumptions are violated.

Non-Parametric Algorithms:

Non-parametric algorithms, in contrast, make minimal assumptions about the functional form of the relationship between the input features and the output labels. They do not have a fixed number of parameters that need to be estimated from the data. Instead, they rely on the training data itself to define the model's behavior. Here are some key characteristics of non-parametric algorithms:

Flexible model structure: Non-parametric algorithms have more flexible model structures that can adapt to the complexity of the data. Examples include decision trees, k-nearest neighbors (KNN), and random forests.

No distribution assumptions: Non-parametric algorithms do not assume any specific distribution for the data. They aim to learn the relationships directly from the data without imposing strong assumptions.

Relatively slower training: Non-parametric algorithms often require more computational resources and time for training since they adapt their model structure to the training data.

Potentially more complex model interpretation: Non-parametric algorithms may offer less interpretable models compared to parametric algorithms due to their flexible and adaptive nature.

Non-parametric algorithms can be more suitable when the relationship between the input features and output labels is highly complex or non-linear, and when there is limited prior knowledge about the underlying data distribution. They can capture intricate patterns but may be prone to overfitting if not properly regularized.

It's important to note that the distinction between parametric and non-parametric algorithms is not always black and white. Some algorithms, like support vector machines (SVMs), have a parametric model but also employ non-linear kernel functions to capture complex relationships, making them a hybrid of the two approaches.

Overall, the choice between parametric and non-parametric algorithms depends on the specific characteristics of the problem, the assumptions about the data, and the desired trade-offs between interpretability and flexibility.